RAG con permisos y búsqueda híbrida · Pre-Entrega 3

Versión pública: la medición se hizo sobre el corpus real de la mesa, que no se publica; el PDF completo se entregó por la plataforma. Las métricas son las mismas; el contenido de la mesa está generalizado.

Forward Deployed Engineer · Coderhouse · Comisión 2026 · Mesa de inversiones de una fintech argentina (nombre reservado)

Matías Gómez · noviembre de 2026

Qué hace: el motor de la PE2 respondía con lo que sabe el modelo; ahora responde con **los documentos de la mesa**: 45 documentos anonimizados que entregó el equipo el 02/11 (manuales de los agentes, el mandato, un playbook de eventos, las reglas, las estrategias, los journals y las revisiones semanales, más exports de órdenes, rechazos y embudo de señales). **Cada usuario recibe solo lo que su rol puede leer**, y si no hay evidencia, el sistema pide una persona en vez de inventar. Es la base del veto del modelo, que es obligatorio desde el mandato v1.1: el veto lee el mandato con este buscador.

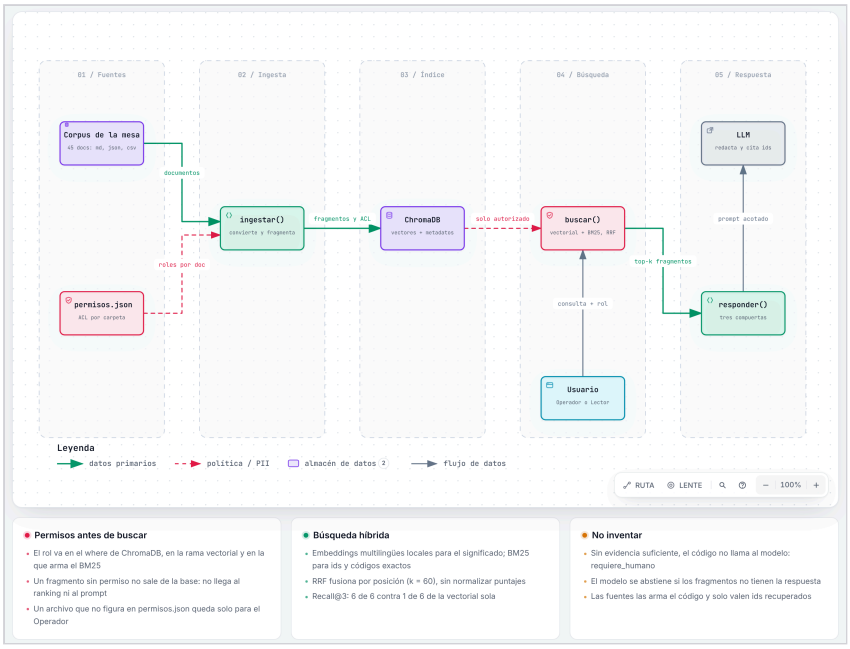


Diagrama hecho con Archify (`entregas/PE3/diagramas/rag-permisos.html`, interactivo). Código en `src/rag/`; 28 tests del RAG, sin red ni claves.

1. Ingesta y metadatos (15 %)

Un solo comando: `python -m src.rag ingestar` lee la carpeta del corpus, limpia, convierte, fragmenta y guarda en ChromaDB: **45 documentos → 1.208 fragmentos en alrededor de 1 minuto**, en una CPU y sin API (embeddings locales). Es idempotente: borra y vuelve a crear la colección.

- **Formatos:** Markdown, texto, HTML, PDF y datos estructurados. Los exports (CSV y JSONL) y el mandato (JSON) pasan a Markdown con un **conversor determinista** dentro de la ingesta: un título por registro con su id (`## P01 · fecha · símbolo`) y una línea con los campos que tienen valor. El JSON anidado pasa a títulos por nivel (`## seccion > subseccion`). Así cada registro es un fragmento citable y su id queda para BM25. El Markdown no se guarda: no hay una copia que se desactualice.
- **Limpieza:** Unicode NFC, espacios y líneas vacías de más; las tablas Markdown pasan a «Encabezado: valor» por fila (una tabla cruda daba un embedding pobre y no se recuperaba). Un archivo roto se informa y no frena a los demás.
- **Confidencialidad:** los documentos traían rutas internas del sistema de la mesa. Se redactaron con etiquetas neutras que conservan fecha e ids de negocio (`[etiqueta · fecha · id]`), y la ingesta **rechaza** cualquier documento que vuelva a traerlas.

Metadatos de cada fragmento (ChromaDB)	
`documento`, `titulo`	archivo dentro del corpus (lo que se cita) y título
`seccion`	ruta de títulos: `Documento > Sección`
`fragmento_indice`, `inicio`, `fin`	posición y rango de caracteres (trazabilidad, small-to-big)
`hash_documento`	sha256: si el documento cambia, se sabe qué invalidar
`rol_operador`, `rol_lector`, `roles`	**quién puede leerlo**: un booleano por rol (filtrable en el `where`) y la lista legible

```
// permisos.json (sale del reparto de permisos que definió la mesa; nombres generalizados)
"agentes/*":      {"roles": ["operador", "lector"]},
"mandato/*":      {"titulo": "Mandato", "roles": ["operador", "lector"]},
"estrategias/*":  {"roles": ["operador", "lector"]},
"exports/*":      {"roles": ["operador"]}
```

2. Permisos antes de buscar (25 %)

Esquema: RBAC (dos roles de la PE1: Operador, que aprueba y ve órdenes, y Lector, que ve el agregado; el veto del modelo lee como Lector) aplicado como una **ACL por documento** (`permisos.json`), que se aplana en cada fragmento al ingestar. No hace falta ABAC (horario, oficina): ningún permiso de la mesa depende de atributos del contexto. Un archivo que no figura en el manifiesto queda **solo para el Operador**: el olvido falla del lado seguro. El mandato y las reglas son de Lector porque el veto los tiene que leer; los exports, de Operador.

El filtro va en la consulta a la base, en las dos ramas. La búsqueda vectorial lleva el `where` de ChromaDB y el índice BM25 se arma solo con los fragmentos que devuelve un `get` con el mismo `where`. Un fragmento sin permiso no sale de la base: no llega a Python, al ranking ni al prompt. No hay camino de búsqueda sin filtro, y nunca se le pide al modelo que «no muestre» algo.

```
filtro = filtro_chroma(usuario)        # {"rol_lector": True}
coleccion.query(query_embeddings=[q], n_results=n,
where=filtro)  # vectorial
visibles = coleccion.get(where=filtro)  # BM25 solo sobre lo
```

Test (`tests/test_rag.py`)	Rol
`test_lector_no_recibe_nada_del_restringido_ni_pidiendo_todo`: la consulta es el texto secreto y se piden 1.000 resultados, en los 3 modos	Lector
`test_lector_sin_permiso_no_recibe_nada_ni_en_el_prompt`: captura el prompt que recibe el modelo; el secreto aparece solo en la consulta, nunca en un fragmento	Lector

```
autorizado
bm25 = BM25Okapi([tokenizar(t) for t in visibles["documents"]])
```

Test (`tests/test_rag.py`)	Rol
`test_operador_si_recibe_el_restringido` y `…_en_el_prompt`	Operador
`test_el_filtro_por_rol_va_en_cada_consulta_a_la_base`: un espía anota cada `query` y `get` y verifica el `where`	los dos
`test_documento_sin_manifiesto_queda_solo_para_el_operador`	—

Con el corpus real y el modelo real: pidiendo todos los resultados, el Lector recibe **0 de 817** fragmentos de los exports. En las 2 preguntas restringidas, el prompt que recibió Haiku 5.5 no tiene ningún fragmento de los exports y la respuesta es «No encontré la respuesta», con `requiere_humano=true`. La misma pregunta, hecha por el Operador, se responde con la cita:

```
$ python -m src.rag responder "¿Qué P&L bruto tuvo la operación P01 del lote de prueba?" --rol lector
  "respuesta": "No encontré la respuesta en los documentos.",  "fuentes": [],  "requiere_humano": true
$ python -m src.rag responder "…la misma…" --rol operador
  "respuesta": "La operación P01 (lote de prueba) tuvo un P&L bruto de [monto] según el registro de la mesa.",
  "fuentes": [{"documento": "[export de operaciones]", "seccion": "[registro P01]", "fragmento_id": "[export de
operaciones]#000"}],
  "requiere_humano": false,  "confianza": 0.58,  "confianza_modelo": 0.95
```

Salida real abreviada; el monto, los nombres de archivo y la sección están generalizados en esta versión.

3. Búsqueda híbrida con evidencia (20 %)

Cómo: dos rankings sobre lo autorizado. El **vectorial** usa `paraphrase-multilingual-MiniLM-L12-v2` (384 dimensiones, multilingüe, local y gratis; coseno en ChromaDB): entiende sinónimos. El **léxico** es **BM25** (`rank-bm25`) con un tokenizador que conserva los códigos enteros y en partes (`reglas-v0.2` → `reglas-v0.2`, `reglas`, `v0`, `2`) y saca palabras vacías: encuentra ids y nombres exactos. Se fusionan con **RRF**: `puntaje = Σ 1 / (60 + posición)` en cada lista, sin normalizar escalas distintas (coseno de 0 a 1, BM25 sin techo). Cada rama trae 20 candidatos.

Consultas de código exacto (un archivo de consultas del repo privado). Se eligieron del corpus **antes** de medir y se informan todas tal como salieron, sin fabricar casos. Acierta si un fragmento del top 3 contiene el código literal; se mide por el texto porque varios códigos aparecen en más de un documento. Resultado con el fragmentado estructural, sobre el corpus redactado. Posición del primer fragmento con el código (— = no está en el top 3):

#	Consulta	Qué es	Vectorial	BM25	Híbrida	Lectura
H1	**`[id de enmienda]`**	**id de una enmienda del mandato**	**—**	**1**	**1**	**la híbrida acierta y la vectorial falla**
H4	**`[CÓDIGO_DE_RECHAZO]`**	**código de rechazo de una orden**	**—**	**1**	**1**	**la híbrida acierta y la vectorial falla**
H3	**`[id de orden]`**	**registro de una orden en un journal**	**—**	**1**	**3**	**la híbrida acierta y la vectorial falla**
H5	`fecha|estrategia|símbolo|precio|stop`	clave de un rechazo (exports)	2	1	1	la híbrida sube del 2.º al 1.º
H6	`[CÓDIGO_DE_ERROR]`	error del planificador de la mesa	1	1	1	empate
H2	`[CÓDIGO_DE_ESTADO]`	estado de otra enmienda	—	1	—	**fallan las dos**: RRF diluye el 1.º de BM25

Lectura honesta: en H2, BM25 pone el fragmento correcto 1.º, pero en la fusión queda detrás de tres fragmentos que la vectorial pone arriba y BM25 también trae más abajo. Es el costo de que RRF pese igual las dos listas. Trabajo futuro, con un set de consultas nuevo: un **RRF ponderado**, o darle prioridad a BM25 cuando la consulta tiene forma de código (sin espacios, con dígitos o `_`). No lo apliqué: lo habría ajustado mirando estas mismas consultas. Va a la PE6.

4. Respuesta con fuentes (15 %)

`responder(entrada, usuario)` recibe la `Entrada` de la PE2 y devuelve `SalidaConFuentes`, que **hereda** de la `Salida` de la PE2 (≈ `Salida.extend({...})` de Zod) y suma `respuesta`, `fuentes`, `requiere_humano` y `confianza_modelo`. El contrato y los tests de la PE2 no cambian. **Tres compuertas para no inventar, y ninguna depende solo del modelo:**

1. **Código, antes del LLM:** si el mejor fragmento visible no llega a similitud 0,40, no se llama al modelo: `requiere_humano=true` y 0 tokens.
2. **Modelo:** el prompt (`rag-prompt-v3`) le pide responder solo con los fragmentos, que van como datos entre etiquetas (defensa contra prompt injection), y abstenerse si no está la respuesta.
3. **Código, después: las fuentes las arma el código desde los metadatos**. El modelo solo cita ids; un id que no estaba entre los recuperados se descarta, y una respuesta sin citas válidas vuelve con `requiere_humano=true`. Una cita mostrada siempre es un fragmento que el modelo recibió.

Lección de la PE2 aplicada: la `confianza` que cuenta la calcula el código con la evidencia (similitud del mejor fragmento citado, +0,15 si también lo encontró BM25). La que declara el modelo queda registrada en `confianza_modelo`, pero no decide nada: en la PE2 cambiaba entre corridas. El RAG tampoco propone operaciones: la acción la fija el código (`no_operar`).

Un error que encontró la medición: con `rag-prompt-v2`, una respuesta correcta y bien citada (P01) salía con `requiere_humano=true`: el documento decía que reactivar «requiere el OK del usuario» y el modelo lo trasladó a su respuesta. La v3 define el campo («habla de tu respuesta, no del contenido») y la pregunta pasó.

5. Preguntas de prueba y medición (15 %)

Las preguntas viven en un archivo del repo privado, sobre el corpus real; son la semilla del set de evals de la PE6. Cada pregunta trae su documento esperado y la respuesta esperada, y el registro de cada consulta (esperado y observado, por configuración) queda en otro archivo. Acá van generalizadas: el tipo, el rol y el resultado son los reales.

#	Tipo	Rol	Pregunta	Documento esperado	Con el LLM (Haiku 5.5, estructural)

P01	normal	Lector	Un límite duro del mandato y qué hace falta para reactivar	mandato	**responde con el valor y la condición**
P02	normal	Lector	Qué condición de mercado habilita una estrategia	ficha de la estrategia	**se abstuvo con fuentes (falla segura)**
P03	normal	Lector	Ante un evento geopolítico, qué se opera y con qué confirmación	playbook de eventos	**responde con los activos y la confirmación**
P04	normal	Lector	¿El agente adversarial puede vetar una idea?	manual de un agente	**responde quién tiene el veto**
P05	normal	Lector	Qué condición de RSI usa otra estrategia	ficha de la estrategia	**responde con la condición**
P06	normal	Lector	A qué precio se llenó una compra de un día y dónde quedó el stop	journal del día	**responde con el llenado y el stop**
P07	sin respuesta	Lector	¿Qué rendimiento tuvo la cartera real del equipo en septiembre?	—	**se abstiene (la frena el código)**
P08	sin respuesta	Lector	¿Qué tasa cobra el bróker por operar con margen?	—	**se abstiene (la frena el modelo)**
P09	restringida	Lector	Según el contrafáctico, ¿cuántos R habría hecho una señal rechazada?	export de rechazos	**nada del export en prompt ni fuentes**
P10	restringida	Lector	¿Qué P&L bruto tuvo la operación P01 del lote de prueba?	export de operaciones	**nada del export en prompt ni fuentes**

Medición top-k con dos configuraciones de fragmentado (`python -m src.rag medir -k 3 --con-llm`). Como en el ejemplo de la clase, **Recall@3 se calcula solo sobre las 6 preguntas con respuesta**; las 2 sin respuesta se validan con la respuesta del LLM y las 2 restringidas, con que el Lector nunca recupera nada del export. El MRR es un dato extra: 1 / posición del primer fragmento del documento correcto.

Fragmentado	Modo	Fragmentos	Recall@3	MRR	10 preguntas con el LLM	Códigos: la híbrida acierta y la vectorial falla
Estructural (900 caracteres, solape de una oración)	**híbrida**	**1.208**	**6/6**	**0,61**	**9/10**	**3**
Estructural	vectorial	1.208	1/6	0,06	—	—
Fijo (300 caracteres, sin solape)	híbrida	2.002	6/6	0,83	9/10	3
Fijo	vectorial	2.002	2/6	0,17	—	—

Lectura: las dos configuraciones híbridas empatan en Recall@3 (6/6) y en las 10 preguntas con el LLM (9/10, en la misma corrida). La vectorial sola casi no sirve en este corpus, lleno de códigos y nombres propios: el documento maestro, largo y general, se parece a todo. En las dos, la pregunta que falla es una **abstención con fuentes** (el modelo no se animó a responder con los fragmentos correctos): es una falla del lado seguro, no un invento. En las 2 restringidas, el Lector nunca recibe nada del export (0 de 817 fragmentos visibles) y el Operador las encuentra en el top 3. La compuerta de similitud frena una sola de las dos sin respuesta (P08 se parece a fragmentos que hablan del bróker): la segunda la frena el modelo, porque parecerse no es contener la respuesta.

La medición es inestable con 6 preguntas: antes de redactar las rutas internas (texto apenas distinto), el fijo daba 5/6 con MRR 0,58 y el estructural, 6/6 con MRR 0,64 y 10/10 con el LLM. Con 10 preguntas, la medición sirve para elegir, no para afirmar una tasa: la PE6 la lleva a 20 casos o más. La evidencia de la corrida, en su versión pública, está en `entregas/PE3/evidencia/`.

6. Decisiones y trade-offs (10 %)

Tamaño de fragmento: 900 caracteres (unas 225 palabras), con el corte por títulos y solape de una oración (hasta 120 caracteres). Es el mismo patrón que el ejemplo de la clase (normalizar a Markdown y cortar por la estructura), con tres mejoras:

1. un tope de tamaño con partición recursiva (párrafo, línea, oración, palabra): una sección larga del documento maestro no queda como un solo fragmento diluido;
2. solape de una oración, para no separar una regla de su excepción;
3. **Contextual Retrieval** con contexto verificable: cada fragmento lleva adelante «Documento > Sección» tomado del propio documento (por ejemplo `[Mandato > límites]`), nunca inventado.

Por qué 900 y no 300, si empatan: con un empate en Recall@3 y en las preguntas con el LLM, y un MRR que cambió de lado entre corridas (0,64 contra 0,58 y después 0,61 contra 0,83), decide lo que no depende del azar. El estructural genera **un 40 % menos de fragmentos** (1.208 contra 2.002: menos embeddings y menos almacenamiento), y cada fragmento es una **sección citable** (regla con sus condiciones, fila del mandato, entrada del journal, registro de un export), mientras que el fijo corta en mitad de una oración y la cita pierde sentido. Con mucho más de 900, el vector promedia temas y el prompt se llena. Si con 20 preguntas el fijo sostiene un MRR mejor, se revisa.

ChromaDB alcanza hoy: 1.208 fragmentos, un proceso, filtros por metadatos con pre-filtering y cero costo; corre en la misma máquina que el bot. **Pinecone (u otra base gestionada) convendría** si (1) el corpus crece a cientos de miles de fragmentos, por ejemplo indexando todos los journals, exports y noticias diarias de la mesa; (2) varios servicios consultan a la vez y hace falta alta disponibilidad, con el bot en la nube y el veto obligatorio de 20 s, donde un índice caído descarta señales; (3) se necesitan namespaces por equipo o cliente con permisos administrados, o (4) infra prefiere no operar el almacenamiento (backups, réplicas). Hoy no se cumple ninguna, y Pinecone sumaría costo, latencia de red y otro proveedor.

Limitaciones: la fusión RRF diluye algunos identificadores (H2); la medición con 6 preguntas no es estable entre corridas; el umbral de similitud (0,40) se calibró con pocos casos; ChromaDB usa un índice aproximado (HNSW), así que una posición puede variar ±1 entre ingestas; los permisos se leen al ingestar, y un cambio de roles exige volver a ingestar (en la PE7, con SSO, el rol vendrá del proveedor de identidad). **Reproducir con el corpus de demo:** desde la raíz del repo, `pip install -r requirements.txt`, `python -m src.rag ingestar --carpeta corpus/demo`, `python -m pytest` y `python -m src.rag buscar "¿cuál es el stop máximo?" --rol lector`. La medición necesita el corpus real y no corre acá. Tag `pe3`.